Математическое моделирование

Лабораторная работа №3
«Имитация систем непрерывных и дискретных случайных величин»

Спецификация программы: теория, функции, графики

Содержание

1 Обзор 2
1.1 Что делает программа 2
1.2 Стек и запуск 2
1.3 Модули 2
1.4 Поток данных 2
2 Теория 4
2.1 Обозначения 4
2.2 Двумерная непрерывная случайная величина 4
2.3 Вариант 1: вывод всех функций 4
2.4 Метод обратных функций и генерация пары 5
2.5 Двумерная дискретная случайная величина 6
2.6 Выборочные характеристики 6
2.7 Распределения, квантили и p-значения 7
2.8 Математическое ожидание: интервал и тест 8
2.9 Дисперсия: эксцесс, интервал и тест 8
2.10 Корреляция 9
2.11 Критерий χ^2 Пирсона на независимость 10
2.12 Гистограммы 10
3 Модули с распределениями и генерацией 12
3.1 `params.py` 12
3.2 `sampling.py` 12
3.3 `discrete_params.py` 13
4 Функции `analysis.py` 14
4.1 Классы-контейнеры 14
4.2 `estimates(sample)` 14
4.3 `_variance_std_error(sample, variance)` 15
4.4 `mean_test(sample, mean0)` 15
4.5 `variance_test(sample, variance0)` 15
4.6 `correlation(x, y)` 15
4.7 `density_histogram(sample, low, high, bins)` 16
4.8 `density_histogram_2d(x, y, x_range, y_range, bins)` 16
4.9 `contingency_table(x, y, values_x, values_y)` 17
4.10 `independence_test_table(observed)` 17
4.11 `class_indices(sample, edges)` 18
4.12 `independence_test(x, y, edges_x, edges_y)` 18
5 Отчёт: `report.py` 19
6 Графики 20
6.1 Вкладка «Continuous 2D»: четыре графика 20
6.2 Вкладка «Continuous 3D» 21
6.3 Вкладка «Discrete 2D»: четыре графика 22
7 Контрольные замечания 25
8 Приложение: краткая сводка формул 26

1 Обзор

1.1 Что делает программа

Программа `lab3` имитирует **пару** случайных величин (X, Y) и выполняет статистическое исследование полученной выборки. Она состоит из двух частей, как и задание.

- **Задание 1.** Двумерная непрерывная случайная величина (НСВ). Вариант № 1 (номер 19 в списке группы: $(19-1) \bmod 9 + 1 = 1$): совместная плотность
$$f(x, y) = e^{-x-y}, \quad x \geq 0,\, y \geq 0.$$
- **Задание 2.** Двумерная дискретная случайная величина (ДСВ) с матрицей распределения 5×5, значения $1, \ldots, 5$ по каждой оси. Матриц две: зависимая и независимая с теми же маргинальными распределениями.

Для каждой пары проверяются независимость составляющих, находятся условные распределения, строятся гистограммы составляющих, считаются теоретические, точечные и интервальные значения математического ожидания, дисперсии и корреляции. Дополнительные (необязательные) пункты тоже сделаны: 3D-гистограмма и проверка гипотез о соответствии оценок теоретическим значениям.

1.2 Стек и запуск

Python, `tkinter` (окно с вкладками), `matplotlib` (графики, в том числе 3D через `mplot3d`), `numpy` (массивы), `scipy.stats` (квантили и функции распределения). Зависимости зафиксированы в `requirements.txt`, окружение лежит в `lab3/.venv`.

```
cd lab3
.venv\Scripts\python main.py          # приложение
.venv\Scripts\python -m unittest      # тесты (20 штук)
```

1.3 Модули

Файл	Роль
`params.py`	Распределение варианта 1: плотности, функции распределения, обратные функции, теоретические M, D, ρ. Чтобы сменить вариант, переписывается только этот файл.
`discrete_params.py`	Две матрицы 5×5, значения x_i, y_j, маргинальные распределения и теоретические характеристики по матрице.
`sampling.py`	Генерация выборки пар: `generate_pairs` (непрерывная), `generate_pairs_discrete` (дискретная).
`analysis.py`	Вся статистика: оценки, доверительные интервалы, проверки гипотез, корреляция, гистограммы, критерии χ^2 на независимость.
`report.py`	Текстовый отчёт справа от графиков (общий для двух заданий).
`task1_tab.py`	Вкладка «Continuous 2D»: четыре графика и отчёт для НСВ.
`task1_3d_tab.py`	Вкладка «Continuous 3D»: 3D-гистограмма и поверхность плотности.
`task2_tab.py`	Вкладка «Discrete 2D»: четыре графика (в том числе 3D) и отчёт для ДСВ, переключатель матрицы.
`main.py`	Окно `tkinter` с `ttk.Notebook` и тремя вкладками.
`test_lab3.py`	Тесты `unittest`.

1.4 Поток данных

Вкладка при нажатии **Generate** (и при выборе другой матрицы) выполняет цепочку:

1. прочитать n из поля ввода и проверить его;
2. `sampling` сгенерировать выборку $(x_k, y_k)_{k=1}^{n}$ из распределения, описанного в `params` или `discrete_params`;

3. `analysis` посчитать гистограммы, таблицу сопряжённости и критерий χ^2;
4. нарисовать графики (`draw`);
5. собрать текстовый отчёт через `report` (`write_report`) и показать его в поле справа.

Предупреждение красным цветом показывается, если ожидаемое число точек в клетке таблицы меньше 5 (см. раздел про χ^2).

2 Теория

2.1 Обозначения

X, Y	составляющие двумерной случайной величины
$f(x,y)$	совместная плотность (НСВ)
$f_X(x), f_Y(y)$	маргинальные (частные) плотности
$f(y\|x)$	условная плотность Y при $X = x$
$F_X, F_Y, F(y\|x)$	соответствующие функции распределения
p_{ij}	вероятность $\mathrm{P}\big(X = x_i, Y = y_j\big)$ (ДСВ)
$\mathrm{M}(X), \mathrm{D}(X)$	математическое ожидание и дисперсия (теоретические)
ρ	теоретический коэффициент корреляции
n	объём выборки
$\overline{x}, s^2, s$	выборочное среднее, выборочная дисперсия, выборочное стандартное отклонение
r	выборочный коэффициент корреляции
α	уровень значимости, здесь $\alpha = 0.05$ (доверие 95%)
ν (или df)	число степеней свободы распределения

2.2 Двумерная непрерывная случайная величина

Совместная плотность $f(x,y)$ задаёт вероятность попасть в область A плоскости:

$$\mathrm{P}((X,Y) \in A) = \iint_A f(x,y)\ \mathrm{d}\ x\ \mathrm{d}\ y, \quad f \geq 0, \quad \iint f = 1.$$

Маргинальные плотности. Чтобы получить плотность одной составляющей, вторую «забывают», то есть интегрируют по ней по всей области, где $f \neq 0$:

$$f_X(x) = \int f(x,y)\ \mathrm{d}\ y, \qquad f_Y(y) = \int f(x,y)\ \mathrm{d}\ x.$$

Интегрируют именно по другой переменной, а не дифференцируют. Проверка: $\int f_X\ \mathrm{d}\ x = 1$.

Условная плотность — отношение совместной к маргинальной (деление, а не интегрирование):

$$f(y|x) = \frac{f(x,y)}{f_X}(x), \quad f_X(x) > 0.$$

Она отвечает на вопрос «как распределён Y, если X уже известен и равен x».

Функции распределения получаются интегрированием плотности от нижней границы области до текущей точки (обязательно с подстановкой нижнего предела!):

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\ \mathrm{d}\ t, \qquad F(y|x) = \int_{-\infty}^{y} f(t|x)\ \mathrm{d}\ t.$$

Независимость. X и Y независимы тогда и только тогда, когда $f(x,y) = f_X(x) f_Y(y)$ для всех x, y. Эквивалентно: $f(y|x)$ не зависит от x.

2.3 Вариант 1: вывод всех функций

Дано $f(x,y) = e^{-x-y}$ при $x, y \geq 0$, иначе 0. Проверка нормировки: $\int_0^\infty \int_0^\infty e^{-x} e^{-y}\ \mathrm{d}\ x\ \mathrm{d}\ y = 1 \cdot 1 = 1$.

Маргинальные плотности. Интегрируем по второй переменной от 0 до $+\infty$:

$$f_X(x) = \int_0^\infty e^{-x-y}\ \mathrm{d}\ y = e^{-x} \int_0^\infty e^{-y}\ \mathrm{d}\ y = e^{-x} \cdot \left[-e^{-y}\right]_0^\infty = e^{-x}, \quad x \geq 0.$$

Аналогично $f_Y(y) = e^{-y}$ при $y \geq 0$. Это экспоненциальное распределение с параметром $\lambda = 1$.

Условная плотность (деление!):

$$f(y|x) = \frac{e^{-x-y}}{e^{-x}} = e^{-y}, \quad y \geq 0.$$

Она не содержит x, значит X и Y независимы; тот же вывод даёт $f(x,y) = e^{-x}e^{-y} = f_X(x)f_Y(y)$.

Функции распределения. Первообразная $-e^{-t}$, подставляем оба предела:

$$F_X(x) = \int_0^x e^{-t}\ \mathrm{d}\ t = \left[-e^{-t}\right]_0^x = -e^{-x} - (-e^0) = 1 - e^{-x}, \quad x \geq 0,$$

и $F_X(x) = 0$ при $x < 0$. Проверка: $F_X(0) = 0$, $F_X(+\infty) = 1$. Так же $F_Y(y) = 1 - e^{-y}$ и $F(y|x) = \int_0^y e^{-t}\ \mathrm{d}\ t = 1 - e^{-y}$ (не зависит от x).

Обратные функции. Нужны для генерации. Решаем $u = F(x)$ относительно x:

$$u = 1 - e^{-x} \quad \Longrightarrow \quad e^{-x} = 1 - u \quad \Longrightarrow \quad x = -\ln(1-u).$$

Итак $F_X^{-1}(u) = F_Y^{-1}(u) = F^{-1}(u|x) = -\ln(1-u)$ при $u \in [0,1)$. Поскольку $1-u$ тоже равномерна на $(0,1)$, можно было бы писать $-\ln u$; в коде оставлено $-\ln(1-u)$.

Теоретические характеристики. По определению $\mathrm{M}(X) = \int_0^\infty xe^{-x}\ \mathrm{d}\ x$. Интегрирование по частям ($u = x$, $\mathrm{d}\ v = e^{-x}\ \mathrm{d}\ x$, $v = -e^{-x}$):

$$\mathrm{M}(X) = \left[-xe^{-x}\right]_0^\infty + \int_0^\infty e^{-x}\ \mathrm{d}\ x = 0 + 1 = 1.$$

Первое слагаемое равно нулю: на нуле множитель x обнуляет его, на бесконечности e^{-x} убывает быстрее, чем растёт x. Второй момент (тем же приёмом, $u = x^2$):

$$\mathrm{M}(X^2) = \left[-x^2e^{-x}\right]_0^\infty + 2\int_0^\infty xe^{-x}\ \mathrm{d}\ x = 2\,\mathrm{M}(X) = 2.$$

Дисперсия $\mathrm{D}(X) = \mathrm{M}(X^2) - (\mathrm{M}(X))^2 = 2 - 1 = 1$. Для Y то же самое. Для независимых величин $\mathrm{M}(XY) = \mathrm{M}(X)\,\mathrm{M}(Y)$, поэтому

$$\mathrm{cov}(X,Y) = \mathrm{M}(XY) - \mathrm{M}(X)\,\mathrm{M}(Y) = 0, \quad \rho = \frac{\mathrm{cov}(X,Y)}{\sqrt{\mathrm{D}(X)\,\mathrm{D}(Y)}} = 0.$$

В `params.py`: `MEAN_X = MEAN_Y = VAR_X = VAR_Y = 1`, `RHO = 0`.

Эксцесс экспоненциального распределения $\kappa = 9$ (центральный четвёртый момент 9 при $\sigma^4 = 1$); он понадобится для интервала дисперсии (раздел про дисперсию).

2.4 Метод обратных функций и генерация пары

Утверждение. Пусть F — непрерывная строго возрастающая функция распределения и U равномерна на $(0,1)$. Тогда $Y = F^{-1}(U)$ имеет функцию распределения F.

Доказательство. $\mathrm{P}(Y < y) = \mathrm{P}(F^{-1}(U) < y) = \mathrm{P}(U < F(y)) = F(y)$: последнее равенство верно, потому что $\mathrm{P}(U < a) = a$ для $a \in [0,1]$, а событие $F^{-1}(U) < y$ равносильно $U < F(y)$ (функция F возрастает).

Генерация пары. Берутся две независимые $u_1, u_2 \sim R(0,1)$ и два шага:
1. $x = F_X^{-1}(u_1)$: обычная одномерная имитация из маргинального закона;
2. $y = F^{-1}(u_2 \mid x)$: обратная функция *условной* функции распределения, x подставляется как параметр.

Так получается пара с совместной плотностью $f(x,y) = f_X(x)f(y|x)$. Если X и Y независимы, $F(y|x)$ не зависит от x и шаги распадаются на две независимые одномерные имитации. Для варианта 1 это ровно так: $x = -\ln(1-u_1)$, $y = -\ln(1-u_2)$.

2.5 Двумерная дискретная случайная величина

Задаётся матрицей $p_{ij} = \mathrm{P}\big(X = x_i, Y = y_j\big)$, где $p_{ij} \geq 0$ и $\sum_{i,j} p_{ij} = 1$. Здесь $x_i = y_j \in \{1, 2, 3, 4, 5\}$.

Маргинальные распределения — суммы по строкам и столбцам:

$$p_{i\cdot} = \sum_j p_{ij}, \qquad q_{\cdot j} = \sum_i p_{ij}.$$

Условное распределение: $\mathrm{P}\big(Y = y_j \mid X = x_i\big) = \frac{p_{ij}}{p_{i\cdot}}$.

Независимость: X и Y независимы тогда и только тогда, когда $p_{ij} = p_{i\cdot} q_{\cdot j}$ для *всех* i, j (все строки матрицы пропорциональны). Достаточно одной клетки с нарушением, чтобы величины были зависимы. Условное распределение $\frac{p_{ij}}{p_{i\cdot}}$ при независимости не зависит от i.

Характеристики:

$$\mathrm{M}(X) = \sum_i x_i p_{i\cdot}, \quad \mathrm{D}(X) = \sum_i \big(x_i - \mathrm{M}(X)\big)^2 p_{i\cdot},$$

$$\operatorname{cov}(X, Y) = \sum_{i,j} \big(x_i - \mathrm{M}(X)\big)\big(y_j - \mathrm{M}(Y)\big) p_{ij}, \quad \rho = \frac{\operatorname{cov}(X, Y)}{\sqrt{\mathrm{D}(X)\,\mathrm{D}(Y)}}.$$

Две матрицы проекта. Зависимая: большая вероятность на диагонали (Y стремится повторять X).

$x \setminus y$	1	2	3	4	5
1	0.090	0.050	0.015	0.010	0.010
2	0.050	0.090	0.050	0.015	0.010
3	0.015	0.050	0.090	0.050	0.015
4	0.010	0.015	0.050	0.090	0.050
5	0.010	0.010	0.015	0.050	0.090

Суммы строк и столбцов: $0.175, 0.215, 0.220, 0.215, 0.175$ (матрица симметрична). Отсюда $\mathrm{M}(X) = \mathrm{M}(Y) = 3$ (симметрия относительно 3), $\mathrm{D}(X) = 2 \cdot (0.175 \cdot 4 + 0.215 \cdot 1) = 1.83$, $\operatorname{cov} = 1.11$, $\rho = \frac{1.11}{1.83} \approx 0.6066$.

Независимая матрица — произведение маргиналов зависимой, $p_{ij} = p_{i\cdot} q_{\cdot j}$. У неё те же M и D (3 и 1.83), но $\operatorname{cov} = 0$, $\rho = 0$. Различие между матрицами — только в зависимости, поэтому по ним удобно показать, что критерий независимости различает случаи.

Генерация (метод отрезков). Интервал $(0, 1)$ делится на отрезки, длины которых равны вероятностям; в какой отрезок попало u, такое значение и выбрано.

1. x_i выбирается по отрезкам длин $p_{i\cdot}$ (границы — накопленные суммы строк);
2. при выбранном i значение y_j выбирается по отрезкам длин $\frac{p_{ij}}{p_{i\cdot}}$ (границы — накопленные суммы строки, делённой на её сумму).

2.6 Выборочные характеристики

Выборочное среднее $\overline{x} = \frac{1}{n} \sum_{k=1}^{n} x_k$ — оценка $\mathrm{M}(X)$.

Выборочная дисперсия

$$s^2 = \frac{1}{n-1} \sum_{k=1}^{n} \big(x_k - \overline{x}\big)^2.$$

Деление на $n - 1$ вместо n — поправка Бесселя. Причина: $\overline{x}$ вычислено по той же выборке и подстроено под неё, отклонения от $\overline{x}$ в среднем чуть меньше, чем от настоящего μ. Точнее, $\mathrm{M}\Big(\sum \big(x_k - \overline{x}\big)^2\Big) = (n - 1)\sigma^2$, значит $\mathrm{M}\big(s^2\big) = \sigma^2$ (оценка несмещённая). В `numpy` это `np.var(sample, ddof=1)`.

Важное различие. D — число, свойство распределения, у вашего X оно равно 1. А s^2 — случайная величина: с каждой новой выборкой она другая, колеблется вокруг D. Разговор про «ошибку s^2» — про размах этих колебаний. $s = \sqrt{s^2}$ — выборочное стандартное отклонение.

2.7 Распределения, квантили и p-значения

2.7.1 Нормальное $N(0,1)$

Стандартное нормальное распределение. Квантиль уровня 0.975 равен $z_{0.975} = 1.96$: вероятность попасть в $(-1.96, 1.96)$ равна 95%. По центральной предельной теореме (ЦПТ) среднее и многие другие выборочные статистики при большом n приближённо нормальны, даже если сама выборка не нормальна.

2.7.2 Распределение Стьюдента t_ν

Если $Z \sim N(0,1)$, $V \sim \chi^2_\nu$ независимы, то $T = \frac{Z}{\sqrt{\frac{V}{\nu}}}$ имеет распределение Стьюдента с ν степенями свободы.

Откуда берётся в задаче. Для нормальной выборки $\frac{\overline{x}-\mu}{\frac{\sigma}{\sqrt{n}}}$ стандартно нормальна. Но σ неизвестна и заменяется на s; величина $(n-1)\frac{s^2}{\sigma^2}$ имеет распределение χ^2_{n-1} и независима от $\overline{x}$. Поэтому

$$T = \frac{\overline{x}-\mu}{\frac{s}{\sqrt{n}}} \sim t_{n-1}.$$

Параметр $\nu = n-1$. Число степеней свободы равно числу независимых слагаемых в s^2. Из n отклонений $x_k - \overline{x}$ свободны только $n-1$: они связаны одним условием $\sum(x_k - \overline{x}) = 0$ (мы уже потратили одну степень свободы на оценку среднего).

Форма. Симметрично, как нормальное, но с более тяжёлыми хвостами: при малом ν значения вдали от нуля встречаются чаще. При $\nu \to \infty$ t_ν сходится к $N(0,1)$. Квантили $t_{0.975,\nu}$ (двусторонний 95%):

ν	1	2	3	5	10	30	100	1000	9999
$t_{0.975,\nu}$	12.706	4.303	3.182	2.571	2.228	2.042	1.984	1.962	1.960

Для $n = 10000$ ($\nu = 9999$) разница с нормальным (1.960) уже в четвёртом знаке: при больших n не имеет значения, брать t или z, при малых — имеет.

2.7.3 Распределение хи-квадрат χ^2_ν

Сумма квадратов ν независимых стандартных нормальных величин: $\chi^2_\nu = Z_1^2 + \cdots + Z_\nu^2$. Среднее равно ν, дисперсия 2ν; распределение несимметричное, принимает значения ≥ 0. Параметр ν — степени свободы. Критические значения на уровне 95% (верхний квантиль 0.95):

ν	1	4	16	24	399
$\chi^2_{0.95,\nu}$	3.84	9.49	26.30	36.42	446.57

Для больших ν квантиль близок к ν плюс немного (поскольку среднее $= \nu$).

2.7.4 Функции `ppf`, `cdf`, `sf`, p-значение и уровень значимости

- `cdf(x)` $= F(x) = \mathrm{P}(T \leq x)$.
- `ppf(q)` — обратная к `cdf` (percent point function, квантиль): `ppf(0.975)` — число, левее которого лежит 97.5% вероятности.
- `sf(x)` $= 1 - F(x) = \mathrm{P}(T > x)$ — функция выживания («хвост справа»). Численно точнее, чем `1 - cdf(x)`, когда хвост очень маленький.

Проверка гипотезы. Формулируют нулевую гипотезу H_0 («истинное значение равно теоретическому»). Считают статистику, у которой при верной H_0 известное распределение. **p-значение** — вероятность получить значение статистики не менее экстремальное, чем наблюдённое, если H_0 верна. **Уровень значимости** $\alpha = 0.05$ — допустимая вероятность ложно отвергнуть верную H_0. Если $p < \alpha$, H_0 отвергают; иначе говорят «не отвергается» (это не доказывает, что H_0 верна, лишь что данные не противоречат ей).

Двусторонний тест. Статистика отклоняется от нуля и в плюс, и в минус, поэтому $p = 2 \cdot \mathrm{P}(T > |t|)$ (в коде `2.0 * stats.t.sf(abs(t), df)`).

Связь с доверительным интервалом. Двусторонняя проверка на уровне α отвергает H_0 тогда и только тогда, когда теоретическое значение лежит вне $(1-\alpha)$-интервала. Поэтому интервал и вердикт теста согласованы.

Ложные отвержения неизбежны: для верной H_0 тест на уровне 5% отвергает её примерно в одном случае из двадцати. Это не ошибка программы (проверено: на 1000 запусках $\overline{x}$-тест и тест дисперсии отвергали верную H_0 в $4.3 - 6.2\%$ случаев).

2.8 Математическое ожидание: интервал и тест

Доверительный интервал (уровень $1-\alpha$):

$$\overline{x} \pm t_{1-\frac{\alpha}{2},\, n-1} \cdot \frac{s}{\sqrt{n}}.$$

Величина $\frac{s}{\sqrt{n}}$ — оценка стандартной ошибки среднего: среднее n величин с дисперсией σ^2 имеет дисперсию $\frac{\sigma^2}{n}$.

Проверка $H_0 : \mathrm{M}(X) = m_0$. Статистика

$$t = \frac{\overline{x} - m_0}{\frac{s}{\sqrt{n}}} \sim t_{n-1} \quad \text{при верной } H_0,$$

p-значение $2 \cdot \mathrm{P}(T_{n-1} > |t|)$. Для ненормальной выборки (у нас экспоненциальная, ДСВ тоже не нормальна) формально t-распределение точно только для нормальных данных, но при большом n по ЦПТ среднее приближённо нормально и тест работает хорошо (это подтверждено: ложные отвержения $\approx 5\%$).

2.9 Дисперсия: эксцесс, интервал и тест

2.9.1 Эксцесс

Эксцесс κ показывает, насколько тяжёлые хвосты у распределения по сравнению с нормальным:

$$\kappa = \frac{\mathrm{M}((X-\mu)^4)}{\sigma^4}.$$

Это четвёртый центральный момент, делённый на σ^4 (чтобы величина не зависела от масштаба). Значения: нормальное $\kappa = 3$, равномерное $\kappa = 1.8$, экспоненциальное $\kappa = 9$. Часто используют *избыточный* эксцесс $\kappa - 3$ (у нормального 0); в программе используется обычный κ.

Выборочная оценка:

$$\hat{\kappa} = \frac{\frac{1}{n}\sum (x_k - \overline{x})^4}{\left(\frac{1}{n}\sum (x_k - \overline{x})^2\right)^2}.$$

Четвёртая степень сильно усиливает редкие большие значения, поэтому при малых n оценка шумная.

2.9.2 Дисперсия оценки s^2

Почему ошибка s^2 зависит от эксцесса. При большом n выборочная дисперсия приближённо равна среднему величин $Z_k = (x_k - \mu)^2$ (замена $\overline{x}$ на μ и $n-1$ на n даёт поправку порядка $\frac{1}{n}$, а колебания s^2 порядка $\frac{1}{\sqrt{n}}$, поэтому поправкой можно пренебречь). Дисперсия среднего из n независимых величин равна $\frac{\mathrm{Var}(Z)}{n}$, а

$$\mathrm{Var}(Z) = \mathrm{M}(Z^2) - (\mathrm{M}(Z))^2 = \mathrm{M}((X-\mu)^4) - \sigma^4 = \kappa\sigma^4 - \sigma^4 = \sigma^4(\kappa - 1).$$

Отсюда

$$\mathrm{Var}(s^2) \approx \sigma^4 \frac{\kappa - 1}{n}, \quad \mathrm{SE}(s^2) = s^2 \sqrt{\frac{\hat{\kappa} - 1}{n}}.$$

Для среднего ошибка $\frac{\sigma^2}{n}$ зависит только от второго момента; для дисперсии усредняются *квадраты*, и ошибка зависит от четвёртого момента. По ЦПТ s^2 при большом n приближённо нормальна.

При нормальных данных $\kappa = 3$, $\mathrm{Var}(s^2) = 2\frac{\sigma^4}{n}$ (точно $2\frac{\sigma^4}{n-1}$): это соответствует формуле через χ^2_{n-1}, у которого дисперсия $2(n-1)$. У экспоненциального $\kappa = 9$, $\mathrm{Var}(s^2) = 8\frac{\sigma^4}{n}$: в 4 раза больше, ошибка в 2 раза.

2.9.3 Почему интервал через χ^2 не подходит

Классический интервал $\left((n-1)\frac{s^2}{\chi^2_{1-\frac{\alpha}{2},n-1}}, (n-1)\frac{s^2}{\chi^2_{\frac{\alpha}{2},n-1}}\right)$ выведен в предположении нормальной выборки и поэтому предполагает $\kappa = 3$. Для экспоненциальных данных он слишком узок. Численный эксперимент (2000 выборок $\mathrm{Exp}(1)$, истинное $\mathrm{D} = 1$):

n	10000	1000	100
покрытие интервала по χ^2	65.4%	68.0%	70.3%
покрытие интервала через $\hat{\kappa}$	94.7%	94.2%	84.9%

Покрытие — доля интервалов, накрывших истинное значение (должно быть $\approx 95\%$). Поэтому в программе используется интервал через эксцесс (вариант B), он не требует нормальности. При малых n (около 100) он тоже занижает покрытие: сама оценка $\hat{\kappa}$ шумная, формула асимптотическая.

2.9.4 Интервал и тест

$$s^2 \pm z_{1-\frac{\alpha}{2}} \cdot \mathrm{SE}(s^2), \quad z_{1-\frac{\alpha}{2}} = 1.96.$$

Проверка $H_0 : \mathrm{D}(X) = \sigma_0^2$: $z = \frac{s^2 - \sigma_0^2}{\mathrm{SE}(s^2)}$, при верной H_0 и большом n приближённо $N(0,1)$, p-значение $2 \cdot \mathrm{P}(Z > |z|)$.

2.10 Корреляция

Коэффициент корреляции $\rho = \frac{\mathrm{cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1]$ измеряет *линейную* связь: ± 1 при точной линейной зависимости, 0 при её отсутствии. Для независимых величин $\rho = 0$, обратное неверно: нулевая корреляция независимости не доказывает.

Выборочная оценка:

$$r = \frac{\sum (x_k - \overline{x})(y_k - \overline{y})}{(n-1) s_x s_y} = \frac{\sum (x_k - \overline{x})(y_k - \overline{y})}{\sqrt{\sum (x_k - \overline{x})^2 \sum (y_k - \overline{y})^2}}.$$

Интервал (преобразование Фишера). Для r нормальное приближение плохое: r ограничено отрезком $[-1, 1]$, у границ распределение скошено. Фишер предложил

$$z = \mathrm{artanh}(r) = \frac{1}{2} \ln\left(\frac{1+r}{1-r}\right).$$

Величина z не ограничена, приближённо нормальна, а её дисперсия почти не зависит от ρ: $\mathrm{Var}(z) \approx \frac{1}{n-3}$ (преобразование «стабилизирует дисперсию»). Поэтому интервал строят для z, а затем возвращают обратным $\tanh$ (монотонная функция, порядок сохраняется):

$$\left(\tanh\left(z - \frac{z_{1-\frac{\alpha}{2}}}{\sqrt{n-3}}\right), \tanh\left(z + \frac{z_{1-\frac{\alpha}{2}}}{\sqrt{n-3}}\right)\right).$$

Формально вывод для нормальных пар, для ненормальных данных (например, зависимая дискретная матрица) интервал приближённый. Для независимых величин ($\rho = 0$) асимптотическая дисперсия r равна $\frac{1}{n}$ при любых распределениях составляющих.

Проверка $H_0 : \rho = 0$:

$$t = r\sqrt{\frac{n-2}{1-r^2}} \sim t_{n-2} \quad \text{при верной } H_0.$$

Две степени свободы теряются на оценку двух средних. Чем больше $|r|$, тем больше $|t|$; p-значение $2 \cdot \mathrm{P}(T_{n-2} > |t|)$.

2.11 Критерий χ^2 Пирсона на независимость

Общая схема. Данные разбиваются на $r \times c$ клеток, в клетку (i, j) попало n_{ij} точек, при верной гипотезе ожидается E_{ij}. Статистика

$$\chi^2 = \sum_{i,j} \frac{\left(n_{ij} - E_{ij}\right)^2}{E_{ij}}.$$

Слагаемое клетки — квадрат отклонения от ожидания в единицах ожидания. Случайные отклонения неизбежны, поэтому $\chi^2 > 0$ даже у верной модели; при верной H_0 статистика приближённо χ^2_ν со средним ν. Если χ^2 больше критического $\chi^2_{1-\alpha,\nu}$, гипотезу отвергают.

Число степеней свободы ν = число клеток минус число ограничений на ожидаемые частоты:
- одно ограничение всегда: $\sum E_{ij} = n$;
- каждый параметр, оценённый *по выборке*, добавляет ограничение.

2.11.1 Непрерывный случай (задание 1)

Каждая ось делится на 20 интервалов *равной вероятности* (классов), границы через обратные функции:

$$F_X^{-1}\left(\frac{i}{20}\right), \quad i = 1, ..., 19, \qquad F_Y^{-1}\left(\frac{j}{20}\right), \quad j = 1, ..., 19.$$

Последний интервал уходит в $+\infty$. Для варианта 1 границы $-\ln\left(1 - \frac{i}{20}\right)$: у нуля интервалы короткие, дальше длиннее, но вероятность попасть в любой из них одна, $\frac{1}{20}$. Все $20 \times 20 = 400$ клеток при верной модели равновероятны, $p_{ij} = \frac{1}{400}$, ожидание $E = \frac{n}{400} = 25$ при $n = 10000$.

Вероятности клеток берутся из теории (границы и F_X, F_Y известны), из выборки ничего не оценивалось, поэтому $\nu = rc - 1 = 399$, критическое значение 446.57 (у верной модели статистика обычно около 399). Проверяется сразу независимость *и* вид маргинальных законов: если χ^2 не отвергает, выборка согласуется с $f = f_X f_Y$ целиком.

2.11.2 Дискретный случай (задание 2)

Здесь проверяется гипотеза «X и Y независимы» для таблицы 5×5. Матрица зависимая, поэтому вероятности клеток для независимости неизвестны, их оценивают по выборке:

$$E_{ij} = \frac{R_i C_j}{n}, \quad R_i = \sum_j n_{ij}, \quad C_j = \sum_i n_{ij}.$$

Оценены 4 свободных параметра для строк (пять вероятностей в сумме дают 1) и 4 для столбцов:

$$\nu = 25 - 1 - 4 - 4 = 16 = (r-1)(c-1).$$

Критическое значение 26.30. У независимой матрицы статистика около 16, у зависимой — тысячи.

Если бы мы проверяли «выборка соответствует матрице p_{ij}» (ожидание np_{ij}), обе матрицы проходили бы тест: выборка берётся ровно из матрицы, с которой сравниваем. Тест независимости так не работает, поэтому он различает таблицы.

2.11.3 Условие применимости

Приближение распределения статистики к χ^2 работает, когда ожидание в клетке не слишком мало. Обычное правило: $E_{ij} \geq 5$. В задании 1: $\frac{n}{rc} \geq 5$, при $n = 10000$ и $r = c = 50$ было бы 4, поэтому взято $r = c = 20$ (25 в клетке). В программе при нарушении показывается красное предупреждение (`MIN_EXPECTED = 5`).

2.12 Гистограммы

Одномерная гистограмма-плотность. Диапазон $[a, b]$ делится на m равных интервалов ширины $\Delta = \frac{b-a}{m}$. Высота столбца

$$h_j = \frac{n_j}{n\Delta}.$$

Смысл: доля точек в интервале $\frac{n_j}{n}$ приближает вероятность $\approx f(x)\Delta$, откуда $h_j \approx f$ на интервале. Площадь столбцов $\sum h_j\Delta$ равна доле точек внутри диапазона (не больше 1; меньше 1 на долю точек вне диапазона). Если нарисовать просто n_j, масштаб не совпадёт с кривой f.

Почему первый столбец ниже кривой. Столбец показывает *среднее* f по интервалу, а красная кривая рисуется в каждой точке. Для убывающей выпуклой f среднее по интервалу меньше значения на его левом крае: для $[0, 0.3]$ и $f(x) = e^{-x}$ среднее $\frac{1-e^{-0.3}}{0.3} \approx 0.864 < f(0) = 1$.

Двумерная гистограмма-плотность. Область $[a_x, b_x] \times \left[a_y, b_y\right]$ делится на $m_x \times m_y$ равных клеток площади $\Delta_x\Delta_y$, высота

$$h_{ij} = \frac{n_{ij}}{n\Delta_x\Delta_y}.$$

Объём всех столбцов равен доле точек, попавших в область. Для варианта 1 при 20×20 клетках на $[0, 6]^2$ клетка у начала координат имеет среднее значение $0.864^2 \approx 0.747 < f(0,0) = 1$. Далеко от начала координат клеток почти пусты: ожидаемое число точек в клетке с левым нижним углом (x_0, y_0) около $n\Delta_x\Delta_y e^{-x_0-y_0-0.3}$ (для $n = 10000$ при $x_0 + y_0 = 6$ это порядка 2 точек), поэтому края шумные, а отдельные столбцы нулевые.

Дискретные частоты. Для ДСВ высота столбца — просто доля $\frac{n_{ij}}{n}$ (интервалов нет): значение сравнивается с p_{ij}.

Эмпирическое условное распределение для НСВ. Точки по x делятся на 3 среза равной вероятности (границы $F_X^{-1}\left(\frac{1}{3}\right), F_X^{-1}\left(\frac{2}{3}\right)$). Для каждого среза строится гистограмма-плотность y только по точкам этого среза, она оценивает $f(y|x)$ для x из среза.

3 Модули с распределениями и генерацией

3.1 `params.py`

Класс `Params` с статическими методами. Все методы векторизованы: принимают числа или массивы `numpy` (нужно, чтобы вычислить плотность на сетке из сотен точек одним вызовом). Ветвление `if x >= 0` для массива не работает, поэтому везде используется `np.where`, и результатом даже для одного числа будет 0-мерный массив (его можно использовать как число).

Вызов	Что делает
`np.where(cond, a, b)`	Поэлементный выбор: где `cond` истинно — значение из `a`, иначе из `b`. Заменяет `if` для массивов.
`np.exp(a)`	Поэлементная экспонента e^a.
`np.maximum(a, b)`	Поэлементный максимум двух массивов/чисел. Здесь `np.maximum(x, 0.0)` заменяет отрицательные x нулём, чтобы не считать e^{-x} с огромным положительным показателем там, где результат всё равно будет отброшен `np.where`.
`np.log(a)`	Поэлементный натуральный логарифм.
`np.ones_like(x, dtype=float)`	Массив из единиц той же формы, что у `x`, типа `float`. Умножение на него «растягивает» результат до формы x.

Теоретические константы: `MEAN_X = MEAN_Y = 1.0`, `VAR_X = VAR_Y = 1.0`, `RHO = 0.0` (вывод в разделе «Вариант 1»).

`f(x, y)` — совместная плотность e^{-x-y} при $x, y \geq 0$, иначе 0:

```
inside = (x >= 0) & (y >= 0)
return np.where(inside, np.exp(-np.where(inside, x + y, 0.0)), 0.0)
```

Внутренний `np.where` подставляет 0 вместо $x + y$ вне области, чтобы `exp` не получал большое положительное число и не давал переполнения; внешний возвращает 0 вне области.

`f_x(x)`, `f_y(y)` — маргинальные плотности e^{-t} при $t \geq 0$, иначе 0 (вывод выше).

`f_y_given_x(y, x)` — условная плотность $f(y|x) = e^{-y}$. Умножение на `np.ones_like(x, dtype=float)` даёт массив формы x, хотя значение от x не зависит (так функция совместима по форме с общим случаем, где зависимость есть).

`cdf_x(x)`, `cdf_y(y)` — $1 - e^{-t}$ при $t \geq 0$, иначе 0.

`cdf_x_inv(u)`, `cdf_y_inv(u)` — $-\ln(1-u)$.

`cdf_y_given_x_inv(u, x)` — обратная к $F(y|x)$, также $-\ln(1-u)$ (умножение на `np.ones_like(x)` для формы).

3.2 `sampling.py`

3.2.1 `generate_pairs(n, rng)`

```
u1 = rng.random(n)
u2 = rng.random(n)
x = Params.cdf_x_inv(u1)
y = Params.cdf_y_given_x_inv(u2, x)
return x, y
```

`rng` — генератор `np.random.default_rng()`; `rng.random(n)` возвращает массив из n независимых равномерных чисел на $[0, 1)$. Так как $u \in [0, 1)$, $1 - u \in (0, 1]$ и логарифм определён. Реализует двухшаговый метод из раздела «Метод обратных функций».

3.2.2 `generate_pairs_discrete(n, matrix, values_x, values_y, rng)`

```
row_sums = matrix.sum(axis=1)
x_bounds = np.cumsum(row_sums)
x_index = np.minimum(np.searchsorted(x_bounds, rng.random(n), side="right"), len(row_sums) -
1)

y_bounds = np.cumsum(matrix / row_sums[:, None], axis=1)
u2 = rng.random(n)
y_index = np.minimum((u2[:, None] >= y_bounds[x_index]).sum(axis=1), matrix.shape[1] - 1)

return values_x[x_index], values_y[y_index]
```

Шаги: суммы строк `row_sums` $= p_{i\cdot}$; накопленные суммы `x_bounds` — правые границы отрезков; `searchsorted(..., side="right")` находит номер отрезка, в который попало u; `np.minimum(..., last)` защищает от выхода за последний индекс (накопленная сумма может оказаться чуть меньше 1 из-за округления). Для y: `matrix / row_sums[:, None]` — условные вероятности строк (`[:, None]` превращает вектор в столбец для деления построчно), `cumsum(axis=1)` — накопленные суммы по строке; `y_bounds[x_index]` — для каждой точки границы её строки (массив $n \times 5$); сравнение `u2[:, None] >= границы` даёт булевую матрицу $n \times 5$, сумма по строке равна числу границ, не превышающих u_2, то есть индексу значения y.

Вызов	**Что делает**
`matrix.sum(axis=1)`	Сумма по оси 1 (по столбцам) — вектор сумм строк. `axis=0` дало бы суммы столбцов.
`np.cumsum(a, axis)`	Накопленные суммы: $a_0, a_0 + a_1, \ldots$.
`np.searchsorted(bounds, u, side="right")`	Для каждого u возвращает индекс, в который его можно вставить в отсортированный `bounds`, сохранив порядок; при `side=\"right\"` — после равных элементов, то есть число границ $\le u$.
`np.minimum(a, b)`	Поэлементный минимум; здесь «обрезка» индекса сверху.
`rng.random(n)`	n независимых равномерных чисел на $[0, 1)$.
`a[:, None]`	Добавляет ось: вектор $(m,)$ превращается в столбец $(m, 1)$ для поэлементных операций с матрицей по правилам broadcasting.
`a >= b`	Поэлементное сравнение, массив `bool`; `True` считается как 1 при суммировании.

3.3 `discrete_params.py`

- `VALUES_X`, `VALUES_Y` $= (1, 2, 3, 4, 5)$ (`np.arange(1, 6, dtype=float)`).
- `DEPENDENT` — зависимая матрица (таблица выше); `INDEPENDENT = np.outer(DEPENDENT.sum(axis=1), DEPENDENT.sum(axis=0))` — матрица произведений маргиналов. `np.outer(a, b)` строит матрицу $c_{ij} = a_i b_j$.
- `MATRICES = {"Dependent": ..., "Independent": ...}` — словарь для переключателя.
- `marginals(matrix)` возвращает `(matrix.sum(axis=1), matrix.sum(axis=0))`, то есть $p_{i\cdot}$ и $q_{\cdot j}$.
- `characteristics(matrix)` возвращает теоретические $\mathrm{M}(X), \mathrm{M}(Y), \mathrm{D}(X), \mathrm{D}(Y), \rho$:

```
mean_x = float(VALUES_X @ p_x)
var_x  = float(((VALUES_X - mean_x) ** 2) @ p_x)
cov    = float((VALUES_X - mean_x) @ matrix @ (VALUES_Y - mean_y))
rho    = cov / np.sqrt(var_x * var_y)
```

Оператор `@` — матричное (скалярное для векторов) произведение. Выражение `a @ M @ b` равно $\sum_{i,j} a_i M_{ij} b_j$, то есть формуле cov из раздела про ДСВ.

4 Функции `analysis.py`

Модуль подключает `numpy as np` и `scipy.stats` (распределения). Константа `CONFIDENCE = 0.95`; уровень значимости $\alpha = 1 - \text{CONFIDENCE} = 0.05$ используется везде.

4.1 Классы-контейнеры

Все четыре класса — `@dataclass(frozen=True)`: декоратор `dataclass` генерирует `__init__`, `__repr__`, `__eq__` по списку полей; `frozen=True` делает объект неизменяемым (после создания поля менять нельзя).

Класс	Поля	Смысл
`Estimates`	`mean`, `variance`, `mean_interval`, `variance_interval`	Оценки среднего и дисперсии с доверительными интервалами (кортежи из двух чисел).
`HypothesisTest`	`statistic`, `p_value`	Результат проверки гипотезы. Свойство `rejected` (`@property`, вызывается без скобок) истинно при `p_value < 1 - CONFIDENCE`, то есть $p < 0.05$.
`Correlation`	`r`, `interval`, `t_statistic`, `p_value`	Оценка корреляции, её интервал Фишера и t-проверка $\rho = 0$.
`IndependenceTest`	`statistic`, `dof`, `critical`, `p_value`, `min_expected`	Результат критерия χ^2: статистика, степени свободы, критическое значение, p-значение, минимальное ожидание в клетке. Свойство `rejected` истинно при `statistic > critical`.

4.2 `estimates(sample)`

Назначение. Точечные оценки $\overline{x}$ и s^2 с 95%-ми доверительными интервалами.

Шаги.

1. `n = len(sample)`; при $n < 2$ вернуть `None` (дисперсия с $n - 1$ в знаменателе не определена).
2. `mean = float(np.mean(sample))`: $\overline{x}$.
3. `variance = float(np.var(sample, ddof=1))`: s^2 с поправкой Бесселя.
4. `dof = n - 1`, `alpha = 1 - CONFIDENCE`.
5. Полуширина интервала для среднего: `stats.t.ppf(1 - alpha/2, dof) * np.sqrt(variance / n)`, то есть $t_{0.975,n-1} \cdot \frac{s}{\sqrt{n}}$.
6. Полуширина интервала для дисперсии: `stats.norm.ppf(1 - alpha/2) * _variance_std_error(sample, variance)`, то есть $1.96 \cdot \mathrm{SE}(s^2)$.
7. Вернуть `Estimates`, интервалы — `(значение - полуширина, значение + полуширина)`.

Вызов	Что делает
`np.mean(a)`	Среднее арифметическое элементов: $\frac{1}{n}\sum a_k$.
`np.var(a, ddof=1)`	Дисперсия: $\sum \frac{(a_k - \overline{a})^2}{n - \mathrm{ddof}}$. Параметр `ddof` (delta degrees of freedom) задаёт, сколько вычесть из знаменателя; по умолчанию `0` (деление на n), здесь `1` — поправка Бесселя.
`np.sqrt(a)`	Поэлементный квадратный корень.
`stats.t.ppf(q, df)`	Квантиль уровня `q` распределения Стьюдента с `df` степенями свободы: число t, для которого $\mathrm{P}(T \le t) = q$. Для $q = 0.975$ даёт множитель двустороннего 95%-го интервала.
`stats.norm.ppf(q)`	То же для стандартного нормального; $q = 0.975$ даёт 1.96.
`float(...)`	Приводит 0-мерный результат `numpy` к обычному `float`.

Пример. `sample = [1, 2, 3, 4]`: $n = 4$, $\overline{x} = 2.5$, $s^2 = \frac{5}{3} \approx 1.667$, $\frac{s}{\sqrt{n}} \approx 0.6455$, $t_{0.975,3} = 3.182$, интервал среднего $2.5 \pm 2.054 = (0.446, 4.554)$. Эксцесс выборки $\hat{\kappa} = 1.64$, $\mathrm{SE}(s^2) = 1.667 \cdot \sqrt{\frac{0.64}{4}} = 0.667$, интервал дисперсии $1.667 \pm 1.307 = (0.360, 2.973)$.

4.3 `_variance_std_error(sample, variance)`

Назначение. Оценка стандартной ошибки s^2 с учётом эксцесса: $\mathrm{SE}(s^2) = s^2\sqrt{\frac{\hat{\kappa}-1}{n}}$ (вывод в разделе про дисперсию). Имя с подчёркиванием — внутренняя вспомогательная функция.

```
centered = sample - np.mean(sample)
kurtosis = np.mean(centered**4) / np.mean(centered**2) ** 2
return variance * np.sqrt((kurtosis - 1.0) / len(sample))
```

`centered**4` — четвёртые степени отклонений, `np.mean` — выборочный четвёртый момент; `np.mean(centered**2)` — выборочный второй момент (в знаменателе в квадрате, получается $\frac{m_4}{m_2^2} = \hat{\kappa}$; здесь второй момент с делением на n, как в определении эксцесса, а не s^2).

Вызов	Что делает
`sample - np.mean(sample)`	Вычитание скаляра из массива поэлементно (broadcasting): центрирование.
`a**k`	Поэлементное возведение в степень.
`np.mean(a)`	Среднее (см. выше).
`np.sqrt(a)`	Корень.

4.4 `mean_test(sample, mean0)`

Назначение. Проверка $H_0 : \mathrm{M}(X) = \mathrm{mean0}$ критерием Стьюдента.

1. При $n < 2$ вернуть `None`.
2. `t_statistic = (np.mean(sample) - mean0) / np.sqrt(np.var(sample, ddof=1) / n)`: $t = \frac{\overline{x} - m_0}{\frac{s}{\sqrt{n}}}$.
3. `p_value = 2.0 * stats.t.sf(abs(t_statistic), n - 1)`: двусторонний $p = 2\,\mathrm{P}(T_{n-1} > |t|)$.
4. Вернуть `HypothesisTest(float(t), float(p))`.

Вызов	Что делает
`np.mean, np.var, np.sqrt`	Как выше.
`abs(t)`	Встроенный модуль числа (нужен модуль, потому что тест двусторонний).
`stats.t.sf(x, df)`	Функция выживания $\mathrm{P}(T > x) = 1 - F(x)$ распределения Стьюдента с `df` степенями свободы.

Пример. `[1, 2, 3, 4]`, $m_0 = 2$: $t = \frac{0.5}{0.6455} = 0.775$, $p = 0.495$, не отвергается.

4.5 `variance_test(sample, variance0)`

Назначение. Проверка $H_0 : \mathrm{D}(X) = \sigma_0^2$ без предположения нормальности.

1. При $n < 2$ вернуть `None`.
2. `variance = np.var(sample, ddof=1)` $= s^2$.
3. `z_statistic = (variance - variance0) / _variance_std_error(sample, variance)`.
4. `p_value = 2.0 * stats.norm.sf(abs(z_statistic))`.
5. Вернуть `HypothesisTest(float(z), float(p))`.

`stats.norm.sf(x)` $= \mathrm{P}(Z > x)$ для $Z \sim N(0,1)$. **Пример.** `[1, 2, 3, 4]`, $\sigma_0^2 = 1$: $z = \frac{1.667-1}{0.667} = 1.000$, $p = 0.317$.

4.6 `correlation(x, y)`

Назначение. Выборочный r, его 95%-й интервал Фишера и t-проверка $\rho = 0$.

1. При $n < 4$ вернуть `None` (в формуле стандартной ошибки $n - 3$ должно быть > 0; для $n = 3$ интервал не определён).

2. `r = float(np.corrcoef(x, y)[0, 1])`.
3. `z = np.arctanh(r)`: $z = \operatorname{artanh}(r)$.
4. `half_width = stats.norm.ppf(1 - (1 - CONFIDENCE)/2) / np.sqrt(n - 3)`: $\frac{z_{0.975}}{\sqrt{n-3}}$.
5. `interval = (np.tanh(z - half_width), np.tanh(z + half_width))`: интервал в исходной шкале.
6. `t_statistic = r * np.sqrt((n - 2) / (1 - r*r))`.
7. `p_value = 2 * stats.t.sf(abs(t_statistic), n - 2)`.
8. Вернуть `Correlation(r, interval, t, p)`.

Вызов	Что делает
`np.corrcoef(x, y)`	Матрица корреляций 2×2: `[[1, r], [r, 1]]`. Элемент `[0, 1]` — коэффициент корреляции Пирсона r между `x` и `y`.
`np.arctanh(r)`	Обратный гиперболический тангенс $\frac{1}{2}\ln\left(\frac{1+r}{1-r}\right)$ (преобразование Фишера).
`np.tanh(z)`	Гиперболический тангенс — обратное преобразование к `arctanh`.
`np.sqrt(a)`	Корень.
`stats.norm.ppf, stats.t.sf`	Квантиль нормального и функция выживания Стьюдента, см. выше.

Пример. $x = 0, \ldots, 9$, $y = 2x + \sin x$: $r = 0.9936$, интервал $(0.9722, 0.9985)$, $t = 24.9$, $p \approx 7 \cdot 10^{-9}$.

4.7 `density_histogram(sample, low, high, bins)`

Назначение. Высоты столбцов одномерной гистограммы-плотности $h_j = \frac{n_j}{n\Delta}$ и границы интервалов.

```
counts, edges = np.histogram(sample, bins=bins, range=(low, high))
width = (high - low) / bins
return counts / (len(sample) * width), edges
```

Точки вне `[low, high]` в `counts` не попадают, но $n =$ `len(sample)` включает все точки, поэтому площадь столбцов равна доле точек внутри диапазона.

Вызов	Что делает
`np.histogram(a, bins, range)`	Разбивает диапазон `range` на `bins` равных интервалов и считает, сколько элементов попало в каждый. Возвращает пару: `counts` (длина `bins`, целые) и `edges` (длина `bins + 1`, границы). Интервалы полуоткрыты $[e_j, e_{j+1})$, кроме последнего: он замкнут справа, значение ровно `high` туда попадает. Значения вне `range` игнорируются.

Пример. `[0.1, 0.4, 0.5, 0.99, 1.0, 7.0]`, `bins=2`, `range=(0, 1)`: `counts = [2, 3]`, `edges = [0, 0.5, 1]` (точки $0.1, 0.4$ в первом, $0.5, 0.99, 1.0$ во втором, 7.0 вне диапазона).

4.8 `density_histogram_2d(x, y, x_range, y_range, bins)`

Назначение. Высоты $h_{ij} = \frac{n_{ij}}{n\Delta_x\Delta_y}$ двумерной гистограммы и границы клеток по обеим осям.

```
counts, x_edges, y_edges = np.histogram2d(x, y, bins=bins, range=(x_range, y_range))
cell_area = (x_edges[1] - x_edges[0]) * (y_edges[1] - y_edges[0])
return counts / (len(x) * cell_area), x_edges, y_edges
```

Вызов	Что делает
`np.histogram2d(x, y, bins, range)`	Двумерный аналог `np.histogram`: делит `range = (диапазон_x, диапазон_y)` на `bins` $\times$ `bins` равных клеток (одно число `bins` для обеих осей) и считает точки в клетках. Возвращает `counts` формы `(bins, bins)`, где `counts[i, j]` — число точек с x в i-м интервале и y в j-м, а также границы `x_edges` и `y_edges`.

Площадь клетки берётся как произведение ширин первых интервалов (все интервалы равны).

4.9 `contingency_table(x, y, values_x, values_y)`

Назначение. Таблица сопряжённости n_{ij} дискретной выборки: сколько раз встретилась пара (x_i, y_j).

```
cells = np.searchsorted(values_x, x) * len(values_y) + np.searchsorted(values_y, y)
return np.bincount(cells, minlength=len(values_x) * len(values_y)).reshape(len(values_x),
len(values_y))
```

Шаги: `searchsorted(values_x, x)` для значения, равного одному из `values_x` (отсортированного массива), возвращает его индекс i; аналогично индекс j по y. Пара превращается в один номер клетки $k = i \cdot c + j$ (число столбцов c = `len(values_y)`). `bincount` считает, сколько раз встретился каждый номер, а `reshape(r, c)` складывает вектор длины rc в матрицу построчно, так что элемент `[i, j]` соответствует $k = ic + j$.

Вызов	Что делает
`np.searchsorted(a, v)`	Для каждого элемента `v` индекс места вставки в отсортированный `a` (по умолчанию `side=\"left\"`: перед равными). Для значения, которое в `a` есть, это индекс самого значения.
`np.bincount(k, minlength=m)`	Считает, сколько раз каждое целое $0, 1, \ldots$ встречается в `k`; результат — вектор длины не меньше `m` (`minlength` гарантирует нули для отсутствующих значений в конце).
`a.reshape(r, c)`	Меняет форму массива без изменения данных, построчная раскладка.

Пример. $x = (1, 2, 2, 3, 1)$, $y = (1, 1, 3, 3, 2)$, значения $(1, 2, 3)$: пары $(1, 1), (2, 1), (2, 3), (3, 3), (1, 2)$, матрица $[[1, 1, 0], [1, 0, 1], [0, 0, 1]]$.

4.10 `independence_test_table(observed)`

Назначение. Критерий χ^2 на независимость для дискретной таблицы; вероятности строк и столбцов оцениваются по самой таблице, $\nu = (r - 1)(c - 1)$.

1. `observed = observed[observed.sum(axis=1) > 0][:, observed.sum(axis=0) > 0]`: отбросить пустые строки и столбцы (для них ожидание ноль и в формуле деление на ноль). Булева маска `observed.sum(axis=1) > 0` выбирает строки с хотя бы одной точкой, вторая — столбцы.
2. `r, c = observed.shape`; если $r < 2$ или $c < 2$, вернуть `None` (таблицы для проверки независимости нет).
3. `n = observed.sum()`.
4. `expected = np.outer(observed.sum(axis=1), observed.sum(axis=0)) / n`: $E_{ij} = R_i \frac{C_j}{n}$.
5. `statistic = float(np.sum((observed - expected)**2 / expected))`: $\chi^2 = \sum \frac{(n_{ij} - E_{ij})^2}{E_{ij}}$.
6. `dof = (r - 1) * (c - 1)`.
7. Критическое значение `stats.chi2.ppf(CONFIDENCE, dof)`, p-значение `stats.chi2.sf(statistic, dof)`, `min_expected = float(expected.min())`.

Вызов	Что делает
`a.sum(axis=0 / 1)`	Сумма по столбцам (`axis=0`, получается вектор длины c) или по строкам (`axis=1`, вектор длины r); без `axis` — сумма всех элементов.
`mask = a > 0`	Булев массив; `a[mask]` выбирает элементы или строки, где `True`. Запись `A[rows][:, cols]` сначала выбирает строки, потом столбцы.
`np.outer(a, b)`	Матрица $c_{ij} = a_i b_j$ (внешнее произведение).
`np.sum(a)`	Сумма всех элементов.
`a.min()`	Минимальный элемент.
`stats.chi2.ppf(q, df)`	Квантиль χ^2_ν уровня `q`; при $q = 0.95$ — критическое значение.
`stats.chi2.sf(x, df)`	Функция выживания χ^2: $\mathrm{P}(\chi^2_\nu > x)$ — p-значение.

4.11 `class_indices(sample, edges)`

Назначение. Номер класса (интервала) каждой точки при заданных границах: `np.searchsorted(edges, sample, side="right")`. С `side="right"` результат равен числу границ, не превышающих значение, то есть номеру интервала $[e_{k-1}, e_k)$ (номер 0 — всё левее первой границы, последний — правее последней).

Пример. границы $[1, 2]$, значения $(-5, 0.5, 1, 1.5, 2, 9)$ дают классы $(0, 0, 1, 1, 2, 2)$. Обратите внимание: значение 1 ровно на границе попало в класс 1 (правый интервал), потому что `side="right"`.

Используется для критерия χ^2 (непрерывный случай) и для разбиения точек на срезы по x на графике условных плотностей.

4.12 `independence_test(x, y, edges_x, edges_y)`

Назначение. Критерий χ^2 на независимость и согласие с теоретическими маргинальными законами для непрерывной выборки; классы равной вероятности, ожидание в каждой клетке $\frac{n}{rc}$, $\nu = rc - 1$.

1. `r = len(edges_x) + 1`, `c = len(edges_y) + 1`: число классов на единицу больше числа границ (для 19 границ — 20 классов). `n = len(x)`.
2. `cells = class_indices(x, edges_x) * c + class_indices(y, edges_y)`: номер клетки $ic + j$.
3. `observed = np.bincount(cells, minlength=r*c)`: вектор длины rc с числом точек в каждой клетке.
4. `expected = n / (r * c)`: одно число, одинаковое для всех клеток (границы выбраны по равной вероятности).
5. `statistic = float(np.sum((observed - expected)**2 / expected))`.
6. `dof = r * c - 1`; критическое значение и p-значение через `stats.chi2.ppf/sf`; `min_expected = expected`.

Функции `np`: `np.bincount`, `np.sum`, `stats.chi2` — см. выше; разностью `observed - expected` из каждого элемента вектора вычитается число (broadcasting).

5 Отчёт: `report.py`

Три функции возвращают списки строк, вкладки склеивают их в текст.

- `component_lines(name, sample, mean0, var0)`: для составляющей — оценка $\overline{x}$ и интервал (`estimates`), проверка $H_0 : \mathrm{M} = m_0$ (`mean_test`), оценка s^2 с интервалом и проверка $H_0 : \mathrm{D} = \sigma_0^2$ (`variance_test`); везде теоретическое значение, оценка, p-значение и вердикт `rejected`/`not rejected`.
- `correlation_lines(x, y, rho0)`: теоретическое ρ, r, интервал Фишера, статистика и p-значение $H_0 : \rho = 0$.
- `independence_lines(title, test)`: статистика, `df`, критическое значение, p-значение, вердикт.

Формат вердикта: `rejected` — гипотеза отвергнута на уровне 5%.

6 Графики

Размер области: `Figure(figsize=(8, 6), tight_layout=True)`, холст `FigureCanvasTkAgg` встроен в `tkinter`. При каждом запуске фигура очищается (`figure.clear()`) и рисуется заново. 3D-оси вращаются мышью (штатная возможность `mplot3d`).

Вызов	Что делает
`np.linspace(a, b, k)`	k равноотстоящих чисел от a до b включительно. Сетка для гладкой кривой плотности.
`np.arange(1, k) / k`	Числа $\frac{1}{k}, \frac{2}{k}, \ldots, \frac{k-1}{k}$ (`arange` — последовательность целых, деление поэлементное). Уровни для границ классов.
`np.diff(edges)`	Разности соседних элементов: ширины столбцов.
`np.meshgrid(a, b, indexing="ij")`	Строит две матрицы координат всех пар (a_i, b_j); с `indexing=\"ij\"` первая индексируется по a (строки), вторая по b (столбцы), как у матрицы n_{ij}.
`a.ravel()`	Разворачивает массив в плоский вектор (построчно).
`np.array([...])`	Создаёт массив из списка.

6.1 Вкладка «Continuous 2D»: четыре графика

Исходные данные: выборка $(x_k, y_k)_{k=1}^{n}$, $n = 10000$ по умолчанию, из `generate_pairs`. Константы: `BINS = 20`, `X_RANGE = Y_RANGE = (0, 6)`, `SLICES = 3`, `SCATTER_POINTS = 2000`. Сетка `figure.subplots(2, 2)` (2×2 осей).

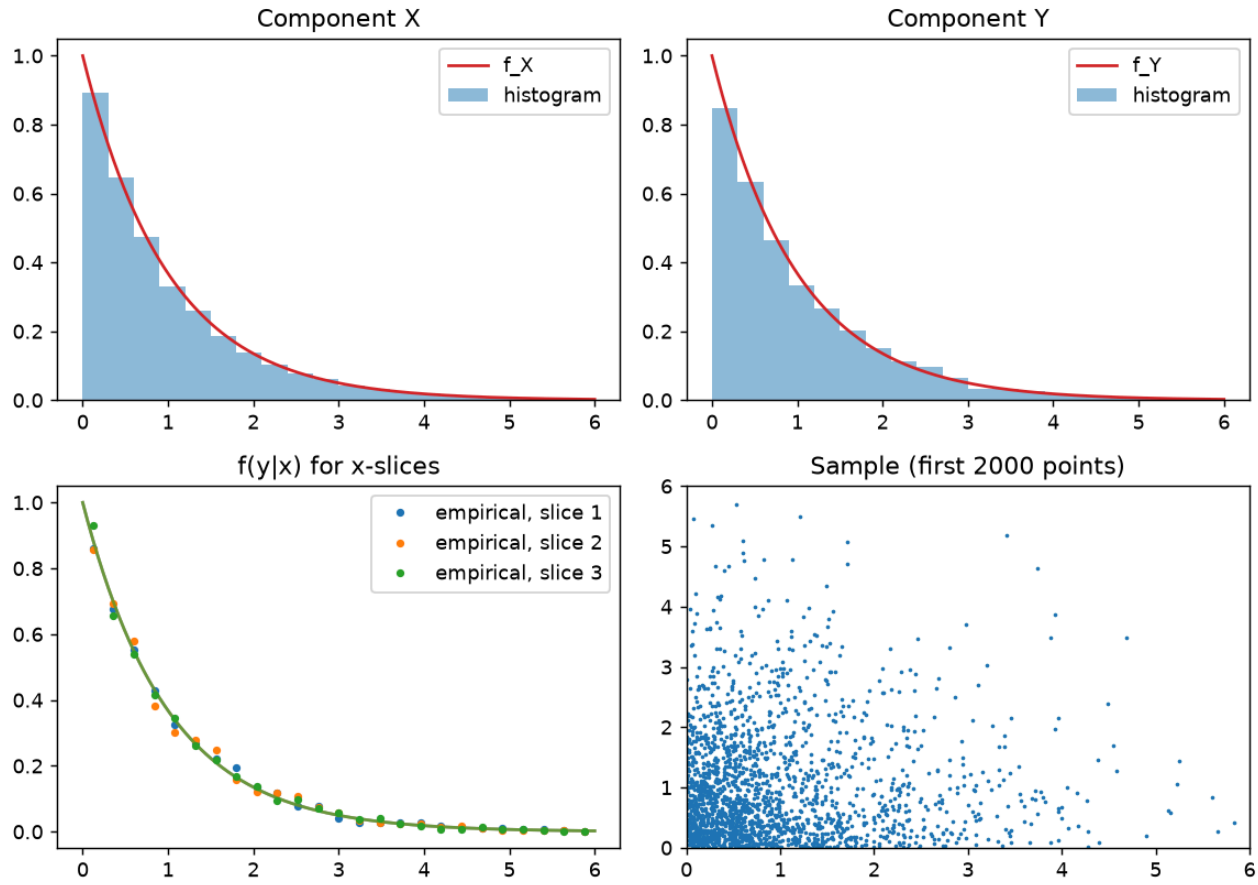


Рис. 1. Вкладка «Continuous 2D»: гистограммы составляющих, условные плотности по срезам, рассеяние.

6.1.1 Component X (слева сверху) и Component Y (справа сверху)

Данные: вектор `x` (соответственно `y`) из всей выборки.

1. `density_histogram(sample, 0, 6, 20)` даёт высоты $h_j = \frac{n_j}{n\Delta}$ и границы 20 интервалов шириной $\Delta = 0.3$.

2. `ax.bar(edges[:-1], heights, width=np.diff(edges), align="edge", alpha=0.5)` — столбцы. `edges[:-1]` — левые края; `width` — ширины; `align="edge"` привязывает столбец левым краем к позиции; `alpha=0.5` — прозрачность.
3. `grid = np.linspace(0, 6, 400)`; `ax.plot(grid, Params.f_x(grid))` — теоретическая плотность e^{-x} красной линией (цвет `"C3"`, четвёртый цвет палитры matplotlib).

Что видно. Если генерация верна, верхушки столбцов лежат около красной кривой. Первый столбец ниже кривой, потому что столбец — среднее f по интервалу, а кривая в начале интервала выше (среднее по $[0, 0.3]$ равно 0.864). На хвосте столбцы низкие и шумные: в них попадает мало точек. За пределами 6 график не показывает значения, их доля около $e^{-6} \approx 0.25\%$.

6.1.2 f(y|x) for x-slices (слева снизу)

Данные: вся выборка, разбитая на срезы по x.

1. Границы срезов: `Params.cdf_x_inv(np.arange(1, 3) / 3)`, то есть $F_X^{-1}\left(\frac{1}{3}\right)$ и $F_X^{-1}\left(\frac{2}{3}\right)$; `class_indices(x, границы)` даёт номер среза $0, 1, 2$ каждой точки. Срезы равной вероятности, в каждом примерно $\frac{n}{3} \approx 3333$ точек.
2. Для каждого среза `s`: `ys = y[slice_of_x == s]` (булев индекс выбирает y точек этого среза), `density_histogram(ys, 0, 6, 25)` — гистограмма-плотность по 25 интервалам (ширина 0.24).
3. `centers = (edges[:-1] + edges[1:]) / 2` — середины интервалов; `ax.plot(centers, heights, ".", color=f"C{s}")` рисует точки (а не столбцы, чтобы три набора не накладывались).
4. Теория: `x_mid = Params.cdf_x_inv(np.array([(s + 0.5) / 3]))` — середина среза по вероятности, то есть x-квантиль уровня $\frac{s+0.5}{3}$; `Params.f_y_given_x(grid, x_mid)` — $f(y|x_{\text{mid}}) = e^{-y}$, линия того же цвета с `alpha=0.6`.

Что видно. Три набора точек лежат на одной и той же кривой e^{-y}, три линии теории совпадают в одну. Это и есть независимость X и Y на картинке: условная плотность Y не зависит от того, в каком срезе x находится точка. Для зависимых величин три облака разошлись бы. Точки шумнее, чем у общей гистограммы: каждый срез содержит только треть выборки.

6.1.3 Sample (справа снизу)

Данные: первые 2000 пар `x[:2000]`, `y[:2000]`. `ax.plot(x, y, ".", markersize=2)` — диаграмма рассеяния, оси ограничены `set_xlim(0, 6)`, `set_ylim(0, 6)`.

Что видно. Облако густое у начала координат (там плотность максимальна) и редеет вправо и вверх. Форма — произведение двух независимых спадов: нет наклона, нет «полосы» вдоль диагонали. Корреляция r в отчёте — число, показывающее, что наклона нет.

6.2 Вкладка «Continuous 3D»

Данные: выборка из `generate_pairs` (свои n и кнопка Generate), `density_histogram_2d(x, y, (0, 6), (0, 6), 20)` даёт `heights` 20×20 и границы `x_edges`, `y_edges`. Доля точек вне $[0, 6]^2$ $1 - \sum h_{ij}\Delta_x\Delta_y$ (показывается над графиком, обычно около 0.5%).

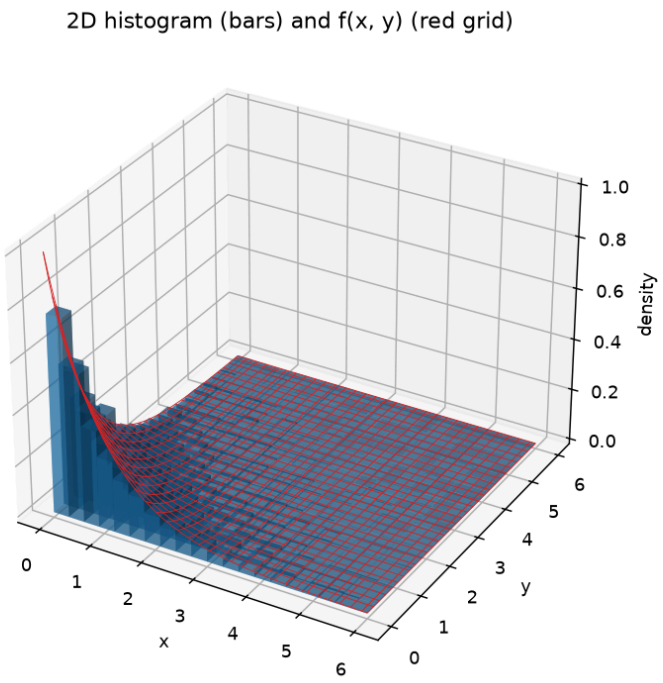


Рис. 2. Вкладка «Continuous 3D»: двумерная гистограмма-плотность (столбцы) и $f(x, y)$ (красная сетка).

Столбцы.

```
xs, ys = np.meshgrid(x_edges[:-1], y_edges[:-1], indexing="ij")
ax.bar3d(xs.ravel(), ys.ravel(), 0.0, dx, dy, heights.ravel(), alpha=0.5, shade=True)
```

`x_edges[:-1]` — левые края клеток по x, `y_edges[:-1]` — по y. `meshgrid(..., indexing="ij")` строит матрицы левых нижних углов всех 20×20 клеток, `ravel()` — плоские векторы длины 400. `bar3d(x, y, z, dx, dy, dz)` рисует параллелепипед с основанием, левым нижним углом в (x, y) и сторонами `dx`, `dy`, снизу с высоты $z = 0$, высотой `dz` = `heights.ravel()` (плоское расположение `heights` совпадает с расположением `xs.ravel()`: оба построчно по (i, j)). `shade=True` включает затенение боковых граней, `alpha=0.5` — прозрачность, чтобы видеть теоретическую сетку сквозь столбцы.

Поверхность (теория).

```
gx, gy = np.meshgrid(np.linspace(0, 6, 60), np.linspace(0, 6, 60), indexing="ij")
ax.plot_wireframe(gx, gy, Params.f(gx, gy), color="C3", linewidth=0.5, rstride=2, cstride=2)
```

Сетка 60×60 точек на $[0, 6]^2$, `Params.f(gx, gy)` — e^{-x-y} в каждом узле. `plot_wireframe` рисует проволочную поверхность: линии вдоль строк и столбцов сетки; `rstride=2`, `cstride=2` берут каждую вторую линию (меньше линий, картинка чище); цвет красный `"C3"`.

Что видно. Столбцы повторяют форму красной сетки: высокие у начала координат, спад к краям. Ось z доходит до 1, потому что в точке $(0, 0)$ сетка равна $f(0, 0) = 1$, а столбец первой клетки показывает среднее по клетке 0.3×0.3, то есть около 0.747. Это эффект ширины клетки, не ошибка. Далеко от начала координат клетки почти пусты, столбцы нулевые, шум на краях — свойство распределения.

6.3 Вкладка «Discrete 2D»: четыре графика

Данные: выборка из `generate_pairs_discrete` по выбранной матрице. Таблица `observed` = `contingency_table(...)` 5×5, `frequencies` = `observed / observed.sum()` — доли $\frac{n_{ij}}{n}$, `p_x`, `p_y` = `marginals(matrix)` — теоретические маргиналы. Переключатель матрицы — `ttk.Combobox` (при выборе график и отчёт пересчитываются).

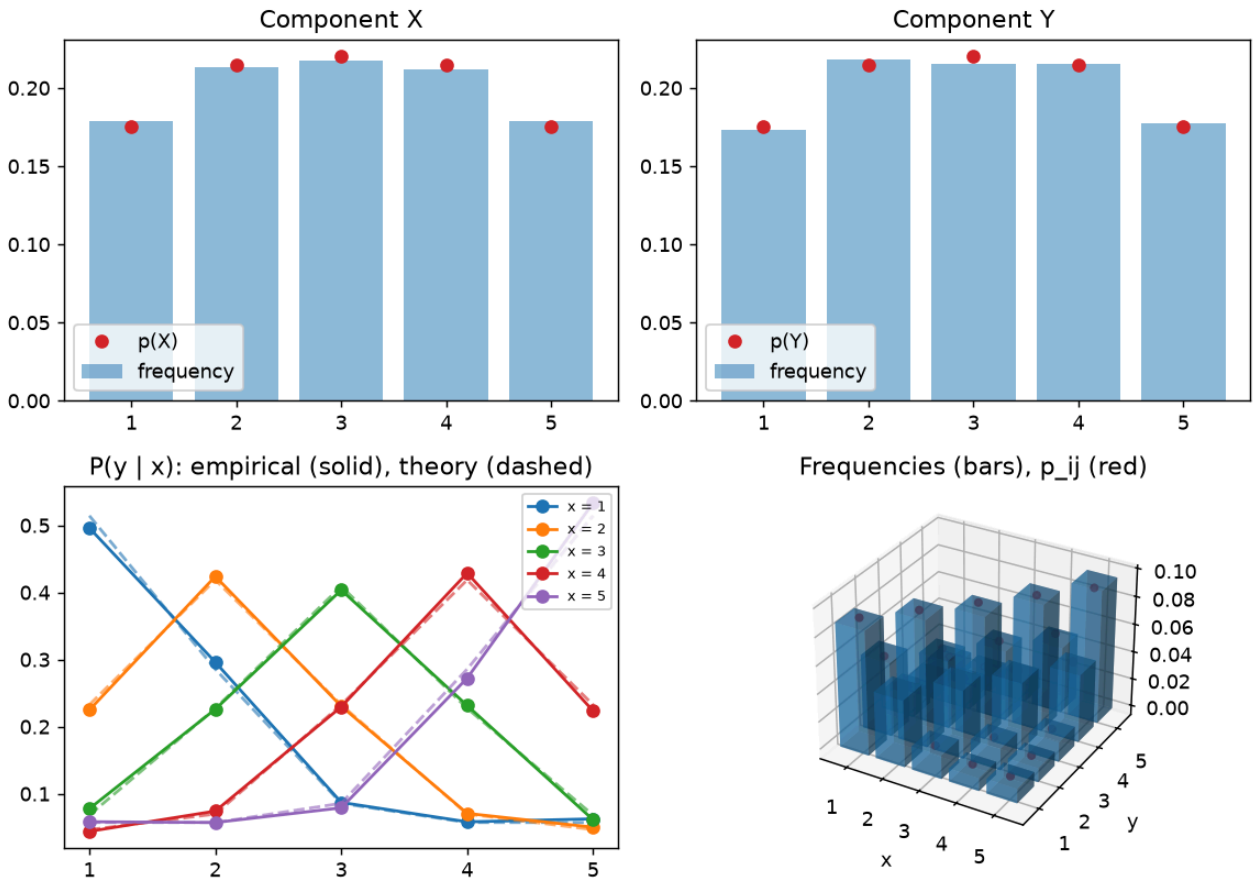


Рис. 3. Вкладка «Discrete 2D», зависимая матрица.

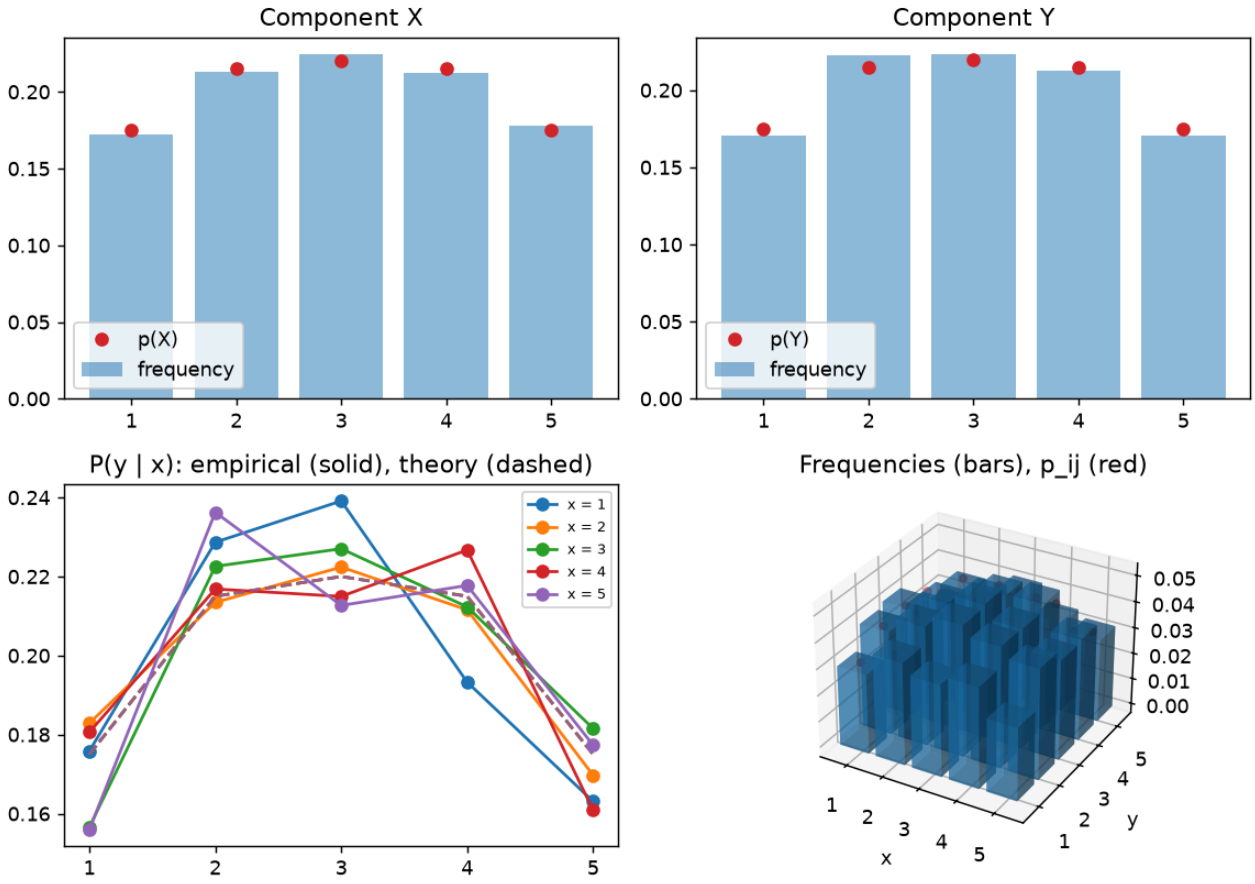


Рис. 4. Вкладка «Discrete 2D», независимая матрица.

6.3.1 Component X и Component Y (сверху)

Данные: `frequencies.sum(axis=1)` — доли по значениям x (сумма строк таблицы частот, эмпирическое маргинальное распределение X), `frequencies.sum(axis=0)` — по значениям y (сумма столбцов).

1. `ax.bar(values, empirical, alpha=0.5)` — столбцы-частоты на позициях $1, \ldots, 5$.
2. `ax.plot(values, theory, "o", color="C3")` — красные точки теоретических $p_{i\cdot}$ (для Y — $q_{\cdot j}$).
3. `ax.set_xticks(values)` — подписи оси ровно в точках $1..5$.

Что видно. Столбцы лежат около красных точек с шумом порядка $\frac{1}{\sqrt{n}}$. У обеих матриц маргиналы одинаковые $(0.175, 0.215, 0.220, 0.215, 0.175)$.

6.3.2 P(y | x) (слева снизу)

Данные: `row_counts = observed.sum(axis=1, keepdims=True)` — число точек для каждого значения x (форма 5×1, `keepdims=True` сохраняет ось для деления построчно). Для $i = 0, \ldots, 4$:

1. эмпирика: `observed[i] / row_counts[i, 0]` — условное распределение y в строке i, то есть $\hat{P}(y_j \mid x_i) = \frac{n_{ij}}{R_i}$; `ax.plot(VALUES_Y, ..., "-o", color=f"C{i}")` — сплошная линия с маркерами;
2. теория: `matrix[i] / p_x[i]` — $\frac{p_{ij}}{p_{i\cdot}}$; `ax.plot(VALUES_Y, ..., "--", color=f"C{i}", alpha=0.6)` — пунктир того же цвета;
3. строки без точек (при малых n) пропускаются.

Что видно. Для зависимой матрицы пять кривых разные: у $x = 1$ максимум в $y = 1$, у $x = 5$ — в $y = 5$ (распределение Y «следует» за X), условные распределения различаются, и сплошные линии идут по своим пунктирам. Для независимой матрицы все пунктиры сливаются в одну кривую $q_{\cdot j}$, а сплошные линии колеблются вокруг неё (шум на $5 \times 5 = 25$ клетках).

6.3.3 3D-гистограмма (справа снизу)

Данные: `frequencies` и `matrix`.

```
xs, ys = np.meshgrid(VALUES_X, VALUES_Y, indexing="ij")
ax.bar3d(xs.ravel() - 0.4, ys.ravel() - 0.4, 0.0, 0.8, 0.8, frequencies.ravel(), alpha=0.5,
shade=True)
ax.scatter(xs.ravel(), ys.ravel(), matrix.ravel(), color="C3", s=12)
```

`meshgrid` строит координаты (x_i, y_j) всех 25 пар. Столбец рисуется шириной 0.8 с левым нижним углом $(x_i - 0.4, y_j - 0.4)$, то есть центрируется на (x_i, y_j); высота — доля $\frac{n_{ij}}{n}$. `ax.scatter` ставит красные точки в (x_i, y_j, p_{ij}) — теоретическую высоту над центром каждого столбца; `s=12` — размер маркера. Для ДСВ нет интервалов, поэтому высота — просто частота, а не плотность.

Что видно. Для зависимой матрицы столбцы высокие вдоль диагонали $x = y$ и низкие в углах; для независимой — ровная «холмистая» поверхность без диагонали. Красные точки находятся над столбцами или у их верхушек.

7 Контрольные замечания

- **Случайность.** Каждое нажатие Generate создаёт новую выборку (генератор без фиксированного seed), поэтому графики и числа слегка меняются. Это нормально.
- **Калибровка тестов.** На 1000 запусках для НСВ: $\overline{x}$-тест для X и Y отвергал верную гипотезу в 4.9% и 4.3%, тест дисперсии — в 5.9% и 6.2% случаев (ожидается 5%). Значит, вердикты `rejected` на верной выборке редки и случайны.
- **Независимость.** На зависимой матрице $\chi^2 \approx 5500$ при критическом 26.3 (отвергается), на независимой ≈ 11 (не отвергается). Для НСВ $\chi^2 \approx 400$ при критическом 446.6 (не отвергается).
- **Корреляция зависимой матрицы.** Теоретическое $\rho = 0.6066$, оценка около $0.59 - 0.61$, интервал иногда не накрывает теорию (в 5% случаев) и из-за приближённости формулы Фишера для ненормальных данных.
- **Малые n.** При $n < 400 \cdot 5 = 2000$ показывается предупреждение для НСВ ($\frac{n}{rc} < 5$); оценка эксцесса при $n < 1000$ шумная, интервал дисперсии тогда менее надёжен.
- **Лаба 2.** Rust-программа лабы 2 использует прежнюю формулу интервала дисперсии (через χ^2); эта спецификация относится только к `lab3`.

8 Приложение: краткая сводка формул

Среднее	$\overline{x} = \frac{1}{n}\sum x_k$
Дисперсия	$s^2 = \frac{1}{n-1}\sum (x_k - \overline{x})^2$
Интервал среднего	$\overline{x} \pm t_{0.975,n-1}\frac{s}{\sqrt{n}}$
Тест среднего	$t = \frac{\overline{x}-m_0}{\frac{s}{\sqrt{n}}} \sim t_{n-1}$
Эксцесс (оценка)	$\hat{\kappa} = \frac{m_4}{m_2^2}$, $m_k = \frac{1}{n}\sum (x_k - \overline{x})^k$
SE дисперсии	$\mathrm{SE}(s^2) = s^2\sqrt{\frac{\hat{\kappa}-1}{n}}$
Интервал дисперсии	$s^2 \pm 1.96 \cdot \mathrm{SE}(s^2)$
Тест дисперсии	$z = \frac{s^2-\sigma_0^2}{\mathrm{SE}(s^2)} \approx N(0,1)$
Корреляция	$r = \sum (x_k-\overline{x})\frac{y_k-\overline{y}}{\sqrt{\sum (x_k-\overline{x})^2 \sum (y_k-\overline{y})^2}}$
Интервал ρ	$\tanh\left(\operatorname{artanh}(r) \pm \frac{1.96}{\sqrt{n-3}}\right)$
Тест $\rho = 0$	$t = r\sqrt{\frac{n-2}{1-r^2}} \sim t_{n-2}$
χ^2 (общий)	$\sum \frac{(n_{ij}-E_{ij})^2}{E_{ij}}$
НСВ	$E = \frac{n}{rc}$, $\nu = rc - 1 = 399$
ДСВ	$E_{ij} = R_i\frac{C_j}{n}$, $\nu = (r-1)(c-1) = 16$
Гистограмма 1D	$h_j = \frac{n_j}{n\Delta}$
Гистограмма 2D	$h_{ij} = \frac{n_{ij}}{n\Delta_x\Delta_y}$
Генерация НСВ	$x = -\ln(1-u_1)$, $y = -\ln(1-u_2)$